

APPLICAZIONI WEB E CLOUD

Simone Paolo Petta

Secondo semestre a.a. 2020/21

Chapter 1

WWW e HTTP

1.1 WWW

E' l'applicazione internet piu' usata definita negli anni '90 da Tim, fornisce le basi per lo sviluppo del servizio web grossomodo rappresenta una rivoluzione perche' diventa un open service, nel tempo tanti hanno provato a controllare l'internet (internet per i poveri zuckemberg) senza riuscirci in quanto internet e' un posto libero e poco moderato questa cosa porta tutta una serie di problemi. Fornisce accesso a contenuto multimediale in quanto le pagine che sviluppiamo sono multimediali, al contrario anni fa era solo testo. Abbiamo i nodi (pagine web) che sono memorizzate e rese disponibili su host e connessi tramite internet.

1.1.1 Storia e statistiche

E' una crescita esponenziale, introdotto negli anni 90, nel 95 supera FTP (un circuito di trasferimento file), nel 2000 tutto viene erogato su web. Viene ormai identificato come un diritto dell'uomo, piu' di 4 miliardi e mezzo di persone hanno accesso al web (ha una penetrazione del 60%). La tecnologia web diventa cosi' importante che erroneamente internet e web vengono confusi.

1.1.2 Caratteristiche

E' l'applicazione internet piu' usata, puo' essere definita come un insieme di ipertesti multimediali distribuiti su molti nodi, fornisce accesso a documenti ipertestuali attraverso la rete internet. E' un sistema distribuito (non esiste solo un unico server) e scalabile, ovvero il protocollo di HTTP a meno di ottimizzazioni nonostante la crescita esponenziale degli utenti, non e' cambiato

1.1.3 Ipertesto

Documento con una struttura non sequenziale. Abbiamo un documento composto da link interni ed esterni che permette di muoversi tra argomenti collegati indipendentemente dall'ordine di presentazione. Esiste solo piu' di testo ma anche hypermedia come immagini, musica, video... Quindi e' cambiato il modo di fruire le informazioni.

1.1.4 Componenti architetturali

Paradigma a ipermedia distribuito:

- Iper: link per riferirsi ad altri documenti
- Media: piu' di semplice testo
- Consiste di pagine web (documenti ipermediali)

Building block del web:

- Web browser: programma applicativo che permette agli utenti di accedere e visualizzare pagine web

- Web server: entita' che fornisce le pagine web al web browser. E' potente, deve garantire parallelismo (ovvero che piu' utenti si colleghino contemporaneamente), sicurezza e privacy.

1.1.5 Architettura

Architettura client-server:

- Il client (web browser) e' un programma che si connette al server, sottomette una richiesta e aspetta una risposta. Il client e' sempre disponibile, non come prima quando il modem era su linea telefonica.
- Il server (web server) e' un programma "in ascolto" (su una porta TCP) che fornisce un servizio.

Data una richiesta del client, il server analizza la richiesta, prepara la risposta, ritorna la risposta al client.

1.1.6 Concetti base

- URL, Uniform Resource Locator e' il nome della pagina web
- HTTP, HyperText Transfer Protocol e' il canale di trasporto che utilizziamo per spostare la richiesta tra client e server
- HTML, HyperText Markup Language

1.1.7 Web Page Identifier

Sono una generalizzazione delle URI:

- Sintassi universale con cui identificare le risorse in rete
- Indipendente dal protocollo
- Basato su un set di caratteri ristretto, per evitare problemi di cattiva codifica
- Non definisce la semantica di una risorsa

Ci permette di identificare una pagina web in modo univoco e senza doppioni.

URL e' un URI che in aggiunta a identificare una risorsa fornisce anche un meccanismo per localizzarla, rispetta la sintassi URI:

- `http:// hostname [:port] / path [? query]` (versione semplificata)

Tendenzialmente gli utenti non vedono i parametri opzionali. Il browser se non e' specificato un documento in particolare cerca sempre un documento `index.html` (o quello che vogliamo).

Formati:

- Assoluto:
 - Usato dagli utenti del web
 - Contiene una specifica completa dell'URL
- Relativo:
 - Significativo solo quando un server e' gia' stato identificato

1.1.8 Web Standard

Trasferimento (HTTP)

1.2 HTTP

E' importante perche' dobbiamo progettare applicazioni che funzionano su HTTP. E' semplice ed intuitivo perche' la semantica del protocollo rimane stabile in tutte le versioni, questa cosa e' sintomo di eccellenza nella progettazione:

- Utente inserisce un'URL o clicca su un link
- Il browser parse l'URL, estrae la informazioni e riceve una copia della pagina
- Il server riceve la richiesta, seleziona la pagina e la ritorna al browser
- Il browser parse la pagina HTML e la mostra all'utente.

Protocollo HyperText Transfer Protocol (HTTP), si basa su TCP che e' un protocollo che permette di trasportare i pacchetti di HTTP, ed e' usato tra browser e Web server. TCP e' la rete autostradale mentre i messaggi che trasportiamo sono dei camioncini che si spostano sull'autostrada.

1.2.1 Fasi

Protocollo di comunicazione per lo scambio di messaggi e ipertesti.

- Set-up della connessione
- HTTP request
- HTTP response
- Connection closed

1.2.2 Caratteristiche

- Protocollo di livello applicativo: i protocolli applicativi si interfacciano all'utente, viene fornito al browser. Assume un protocollo di trasporto affidabile e orientato alla connessione, non fornisce affidabilità e ritrasmissione.
- Paradigma richiesta risposta: la comunicazione inizia con una richiesta HTTP
- E' un protocollo STATELESS (senza stato): il server non mantiene lo stato della connessione. Non c'è collegamento tra due richieste consecutive, sono gli sviluppatori che creano uno stato tra le varie richieste HTTP per garantire una sessione fatto tendenzialmente con i cookie.
- Permette trasferimenti bi-direzionali: ad esempio le form, trasferiamo contenuti su server.
- Negoziamenti delle capability: il mittente specifica le sue capability e il ricevente le accetta tutte o in parte. Ad esempio siti web con preferenze sulla lingua che può essere fatta con la locazione o con richieste HTTP. Io ti chiedo qualcosa e tu me lo ridai se ce l'hai o se vuoi darmelo.
- Supporto per il caching: è la memorizzazione delle risorse web per il loro riutilizzo. Diminuisce il tempo di risposta per richieste alla stessa pagina, il browser determina se la pagina è cambiata.
- Supporta intermediari nella comunicazione: supporta il proxy, ad esempio in un'università possiamo accedere e beneficiare degli account dell'università per librerie, il computer è riconosciuto nella rete dell'università e tramite proxy possiamo accedere a siti senza essere loggati.

1.2.3 Connessioni HTTP

- Accede all'URL
- Estrae l'hostname
- Contatta il DNS per ottenere l'indirizzo IP
- Crea una connessione TCP con il server
- Una volta che la connessione

Per questo è importante il caching, in quanto queste operazioni sono molto dispendiose per il computer.

1.3 HTTP/1

1.3.1 Operazioni

- GET
- POST : andiamo a spedire del contenuto
- HEAD: per avere info sull'oggetto stesso (ad esempio sulla data di ultima modifica). Utile per il caching.
- PUT: trasferisce un documento al server, da 1.1
- DELETE: Elimina un documento dal server, da 1.1

GET URL HTTP/version number. Per fare qualche prova di visualizzazione possiamo usare CURL che ci permette di fare delle richieste ad un server web, con -I facciamo una richiesta di HEAD e ci restituisce tutti i metadati della pagina per capire come e' fatta la pagina.

1.3.2 HEADER

E' composto da coppie attributo/valore. Esistono HEADER di richiesta o di risposta

Campi di Header della risposta:

- ok 200
- Moved 301
- Not Modified 304
- Bad Reques 400
- Forbidden 403
- Not found 404
- Internal error 500

1.3.3 Richiesta GET

Quando facciamo una GET di un certo indirizzo otteniamo in risposta una pagina, con una parte di intestazione che ci dice come e' andata la richiesta, poi ci sono i campi dell'header ed infine la pagina web.

In HTTP 1.0 creiamo una connessione TCP per ogni scambio, mentre in HTTP 1.1 piu' trasferimenti viaggiano su una stessa connessione TCP creando una connessione persistente.

1.3.4 Connessioni persistenti

Consentono di creare una sola connessione TCP finché non c'è nell'header `connection closed`. Permette un overhead ridotto, meno connessioni TCP creano latenza della risposta più bassa, meno memoria per i buffer e minor tempo di CPU usato, richieste vengono messe in pipeline, le richieste vengono mandate una dopo l'altra senza aspettare la risposta. Gli svantaggi è che bisogna identificare inizio e fine di ogni oggetto inviato sulla rete internet:

- Mandare una lunghezza seguita da un oggetto
- Inviare un valore sentinella

1.3.5 Persistenza

Pagine web dinamiche non permettono di calcolare la dimensione della pagina in anticipo. Salvare i dati su un file per calcolarne la dimensione è poco pratico. Server chiude la connessione dopo aver mandato i dati. Quello che si fa con HTTP 1.1 si fa a pezzetti il messaggio (chunking) il chunk di dimensione 0 ci dice che l'oggetto è stato inviato completamente.

1.3.6 Negoziazione

Gli HTTP header sono usati per negoziare capability, l'header è usato per stabilire il formato della pagina. Riguardano:

- Connessione (autenticazione)
- Rappresentazione (jpeg, quale tipo di compressione)
- Contenuto (il linguaggio)
- Controllo (validità della pagina)

Guidata dal server, in questo caso abbiamo la versione più efficiente:

- Negoziazione inizia con una richiesta del browser
- La richiesta contiene URL e capability
- Il server seleziona la capability che soddisfa le sue preferenze o la migliore

Guidata dal client:

-

Noi siamo in grado di specificare il livello di preferenza, il browser può specificare rappresentazioni che sono accettabili con un valore di preferenza per ognuna. `Accept: text/html, text/plain;`

1.3.7 Proxy server

Da l'idea di mettere un punto centrale in cui tutti quelli collegati alla rete entrano ed escono. Ad esempio in un momento in cui siamo collegati noi usciamo come università. Noi anche fuori dall'università possiamo settare il proxy e quindi entrarci come se fossimo collegati alla rete universitaria. Forniscono un'ottimizzazione con carico e latenza ridotti, il browser può essere configurato per contattare un proxy. Il proxy configurator per mantenere una cache di pagine web, ci permette di ottimizzare i tempi di accesso a pagine. Se noi passiamo dal proxy ricordiamoci che i log della nostra navigazione entrano nel proxy.

1.3.8 Caching

L'obiettivo è quello di ridurre il traffico. Il problema è stabilire la data di scadenza per la pagina. Quando riutilizzo una pagina piuttosto che andarla a chiedere? Se utilizziamo un proxy passiamo da cache locale a cache condivisa.

Chaching di pagine web

È il server che ci dice se possiamo memorizzare la risorsa in cache e per quanto tempo, non va lasciato il compito al browser. Il server specifica:

- Se la pagina può essere memorizzata in cache
- Il tempo massimo per cui la pagina può essere tenuta in cache

Memorizzazione nella cache

I browser memorizzano le pagine nella cache per risparmiare banda. Se riusciamo ad accedere alla cache ne abbiamo benefici in termini di tempo e memoria, in oltre ne guadagniamo anche in termini di banda riducendo il traffico. Mantengono una memorizzazione temporanea per le pagine recenti. Quando è richiesta una pagina, il browser controlla per vedere se è già nella cache, se non lo è genera la richiesta GET, quando arriva il messaggio di risposta, il browser visualizza la pagina e la memorizza nella cache (insieme ad informazioni dell'header), se abbiamo un proxy nel mezzo questo flusso di richieste e risposte viene applicato a tutti gli utenti. La cache è controllata dai response header:

- Cache-control: possiamo memorizzarlo in cache? è privata o pubblica? Ci presenta le opzioni di controllo della cache. No-cache, private(solo per l'utente), public(in un proxy nella cache pubblica è accessibile da chiunque)
- Last modified: ci permette di gestire una richiesta condizionale, se la pagina è più vecchia di una determinata data allora non spedirla. Mandiamo una richiesta basata sull'ultima modifica della pagina

- Expires: ci dice la data di scadenza, quindi possiamo utilizzare quella pagina fino alla scadenza dell'oggetto stesso.

Se la pagina e' gia' presente nella cache, il browser invia la richiesta GET con l'header If-modified-since relativo ai dati della pagina, se ritorna 200 il browser visualizza la pagina e la memorizza nella cache, se il codice di stato e' 304 visualizza la versione memorizzata nella cache che significa che la pagina non e' cambiata. Queste sono richieste condizionali che ci permettono di controllare la copia in cache per vedere se e' valida e nel caso in cui la pagina non sia stata aggiornata usiamo quella in cache, altrimenti se e' stata modificata viene inviata.

1.4 HTTP/2

Attorno al 2012 il traffico aumenta tantissimo ed il servizio va un po' in sofferenza non tanto per la banda ma perche' andiamo a saturare con ad esempio la qualita' dei contenuti multimediali. I problemi di HTTP 1.1:

- Concurrent connection limit (limite di connessioni concorrenti): si pone un limite a livello teorico al numero di connessioni che possono connettere uno stesso client ad uno stesso server.
- Head of line blocking: In alcuni scenari ci sono dipendenze tra oggetti che non possono essere spezzati, quindi se la richiesta si blocca, tutte le altre aspettano quindi non c'e' pipelining.
- TCP slow start: quando facciamo partire una connessione deve andare a convergenza (non parte al massimo), parte piano e a poco a poco occupa tutta la banda che ha a disposizione per evitare sovraccarichi della rete, questo porta dei grandi svantaggi nel caso in cui dobbiamo fare tante connessioni

L'impatto maggiore per il download di una pagina non e' la banda ma la latenza.

HTTP/2 consente di utilizzare in modo piu' efficiente le risorse di rete e di ridurre la percezione della latenza, introduce compressione del campo header e consente piu' scambi simultanei sulla stessa connessione. I problemi vengono ottimizzati in una soluzione che permette di intervallare (multiplexing) messaggi di richiesta e di risposta sulla stessa connessione e utilizza una codifica efficiente per i campi di intestazione HTTP, inoltre consente di prioritizzare le richieste, permettendo di completare piu' rapidamente le richieste piu' importanti, migliorando ulteriormente le prestazioni. Al contempo mantiene la semantica di HTTP/1.1 (metodi, codici di stato, campi degli header, URI). Il risultato e' che si ottengono meno connessioni TCP, c'e' meno concorrenza con altri flussi di dati e connessioni a lungo termine, migliore utilizzazione della capacita' dei rete disponibile, elaborazione piu' efficiente dei messaggi tramite l'uso di framing dei messaggi binari.

1.4.1 Caratteristiche

Implementiamo una gestione delle risorse ottimizzata nativamente nel protocollo. Il messaggio di HTTP/2 viene diviso in frame

Binary Framing Layer

Cambiamo la codifica del dato, prendiamo i messaggi li suddividiamo e li scambiamo in formato binario. Abbiamo 3 componenti principali:

- Stream: sono le quantità di informazioni che fluiscono tra client e server, può passare uno più messaggi
- Messaggio: è una richiesta logica, una sequenza completa di frame che mappano a una richiesta logica o un messaggio di risposta
- Frame: l'unità singola di comunicazione, al minimo identifica il flusso, ogni frame contiene una intestazione di frame, un numerino che identifica il messaggio

Ogni connessione TCP contiene tutte le comunicazioni con tanti stream, un identificativo univoco più informazioni di priorità opzionali. Il frame è la più piccola quantità di dati scambiabile.

Multiplexing

Ogni stream di richiesta e risposta ha un ID e contiene diversi frame (header, data...) quindi posso spedire più stream insieme, posso anche dare priorità a un particolare stream o frame.

- Richieste in parallelo senza bloccare nessuno
- Risposte in parallelo senza bloccare nessuno
- Singola connessione per più richieste e risposte in parallelo
- Ridurre i tempi di caricamento della pagina eliminando la latenza e migliorando

Ogni stream ha un peso una dipendenza con i quali creo una priorità tra gli stream.

Compressione dell'header

L'header è pochi bit (una serie di coppie attributo valore) in realtà aggiungo 500/800 bit di overhead. HTTP/2 introduce il formato di compressione HPACK per compattare l'header. Questo avviene tramite due tabelle:

- Tabella statica: elenco di header HTTP comune che tutte le connessioni possono utilizzare (rapporto 10:1), indice = una stringa

- Tabella dinamica: inizialmente vuota e aggiornata in base ai valori scambiati all'interno di una connessione, quindi se abbiamo già mandato alcune campi dell'header quei campi vengono aggiunti in coda alla tabella di mapping prende il primo indice libero e quando dobbiamo spedire nuovamente il messaggio mettiamo solo l'indice scelto in precedenza.

Server Push

Quando un client e server comunicano e c'è richiesta il server sa già che tipo di dati verranno richiesti dal client. Tipicamente, dopo aver inviato l'HTML, il server aspetta richieste per altre risorse. In HTTP/2, il server fa un push delle risorse comuni:

- JS, CSS, Immagini, pagine HTML per navigazioni future
- PUSH_PROMISE indicano l'intenzione del server di eseguire push delle risorse descritte al client e devono essere consegnate prima dei dati. Invio queste promesse perché il client può rifiutare un invio (invia solo gli header) nel caso l'elemento sia già in cache.

Abbiamo guardato nel tool di sviluppo del browser (network) che HTTP/1 usa tanti connection ID diversi (connessioni parallele) mentre HTTP/2 ne usa solo uno perché manda sulla stessa connessione più oggetti.

1.5 HTTP/3

Siamo già ad HTTP/3 stanno cambiando il protocollo di trasporto facendo in modo che supporti nativamente il multiplexing, ma mantiene comunque la sintassi di HTTP/1. Nonostante sia molto recente raggiunge già una buona fetta di mercato (più del 10% dei server attuali).

Chapter 2

HTML

2.1 Introduzione

L'HTML **NON** e' un linguaggio di programmazione ma e' un **linguaggio di markup** modellato per rendere esplicita una particolare interpretazione di un testo, che viene interpretato dall'interprete del browser. Oltre a rendere il testo piu' leggibile permette di specificare ulteriori usi. Ad esempio immagini, colori, suoni etc. Non e' presente inoltre nessun meccanismo di decisione ed iterazione e ci sono poche e semplici regole sintattiche. Con markup specifichiamo le modalita' esatte di utilizzo del testo nel sistema stesso, attraverso l'utilizzo di tag. Questi sono linguaggi che favoriscono il riutilizzo e generalmente sono di semplice apprendimento. L'acronimo e' HyperText Markup Language e' usato principalmente per descrivere la struttura della pagina

2.1.1 Caratteristiche

HTML e' un linguaggio per la creazione di pagine web ed e' lo standard per la rappresentazione delle stesse, descrive la struttura della pagina e i dati, le regole e come mostrarli. Ha poche e semplici regole sintattiche, questo implica che il browser fa best effort, ovvero prova a visualizzare il meglio possibile della pagina anche se dovessero esserci errori nel codice HTML. Le caratteristiche principali sono:

- Iper testo
- Multimedia
- Ipermedia

2.2 Browser

E' il programma preposto alla navigazione nel web, richiede risorse attraverso il web e le mostra nella finestra inoltre legge ed interpreta i documenti HTML, CSS, JavaScript ed immagini sulla base di specifiche e standard. Il suo obiettivo principale e' quello di interpretare la pagina, in piu' esegue degli script (js eseguiti lato client) Le componenti principali del browser sono:

- Interfaccia: controlli del browser per controllarlo
- Browser engine: permette di interpretare la pagina
- Rendering engine: riceve il codice HTML e lo interpreta
- JavaScript engine: ha un interprete che interpreta gli script js
- networking: gestisce le funzioni di rete, ad esempio le richieste HTTP
- UI backand: Contiene lo storage, una porzione di memoria dove memorizza gli oggetti

Non tutti i browser sono uguali, al 99 % lo sono ma tendono a farsi dei piccoli dispetti tra di loro, quindi bisogna mettersi in testa di avere una mentalita' multi-browser, ovvero cercare di rendere visibile la pagina web sulla maggior parte dei browser. Contiene anche i tool di sviluppo, possiamo aggiungere delle estensioni.

2.2.1 Caratteristiche

Deve concentrarsi sul rendering, ovvero come il browser rappresenta la pagina web. Inoltre contiene tool per sviluppatori che sono molto utili per fare debugging. Attraverso questi tool siamo anche in grado di manipolare la pagina web in locale.

2.3 Standard web e XHTML

L'HTML e' uno standard del W3C. Dall'HTML nasce l'XHTML, le differenze tra i due non sono cosi' marcate.

2.3.1 Storia degli standard web

Ogni linguaggio di programmazione per il web passa queste fasi di verifica e revisione del W3C. Le fasi sono:

- Working Draft(WD) : e' un documento che il W3C pubblica per ottenere una revisione dalla comunita'.
- Candidate Recommendation (CR) : Documento revisionato in dettaglio che il W3C ritiene soddisfi i requisiti tecnici del Group.
- Proposed Recommendation (PR): Un technical report maturo che, dopo una dettagliata revisione che valuta l'implementabilita' e la correttezza della soluzione, viene inviato per la revisione finale.
- W3C Recommendation (REC): Una specifica o insieme di linee guida che, dopo aver raccolto un ampio consenso, ha ricevuto l'approvazione.

Se noi vogliamo utilizzare il web dobbiamo attenerci ai suoi standard.

2.4 Scrivere una pagina web

E' un file di testo che creiamo con estensioni .html oppure .htm, per crearlo dobbiamo usare un editor testuale oppure un editor visuale, dopo averla scritta possiamo visualizzare la pagina web.

2.5 Struttura della pagina HTML

2.5.1 Tag

I tag sono marcatori che identificano porzioni di testo che vengono poi personalizzate, vengono prima di tutto strutturate e poi vengono formattate e rappresentate. Il nome del tag = nome della funzione, il compito dei tag e' circa il compito delle funzioni in un linguaggio di programmazione. Il tag e' cosi' composto <tag attributi>contenuto</tag>, come dicevamo la sintassi e' leggera, quindi ad esempio se dovesse mancare il tag di chiusura il browser comunque cerchera' di compilare la pagina web. Esempio: testo oppure senza contenuto

Struttura tag

- Tag contenuto tra parentesi triangolari
- tag di inizio e di fine
- contenuto del tag

2.5.2 Struttura della pagina

- riga di intestazione : <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//IT">, che va ad indicare quali sono le specifiche del W3C che vengono utilizzate nella pagina e quali linguaggio stiamo utilizzando. HTML5 la semplifica.
- Envelop (contenitore) : <html> ...codice HTML... </html>, sono i tag di chiusura ed apertura del file HTML.
 - Testa : andiamo a descrivere quali sono i metadati ed informazioni che descrivono il documento, da informazioni su come il documento dev'essere letto ed interpretato. Contiene inoltre meta-tag, script, stili ed e' racchiuso tra il tag <head> ... </head>.
 - Corpo : contiene il documento vero e proprio e tutti i tag per la realizzazione del sito Web (ad esempio <font...>). Anche lui e' racchiuso in tag <body> ... </body>.

Il primo esempio di pagina html, rappresenta una pagina HTML con i tag essenziali (html, head, body) inserendo una stringa all'interno del corpo:

```

1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//IT">
2 <html>
3 <head>
4 </head>
5
6 <body>
7 Hello World!
8 </body>

```

```
9  
10 </html>
```

2.5.3 Indentazione

E' buona norma utilizzare caratteri di tabulazione, aumenta la leggibilita' e diminuisce i tempi di modifica. E' fondamentale usare una buona indentazione perche' una pagina HTML su una stessa riga risulterebbe illeggibile. Il web evolve in continuazione quindi le pagine web vengono modificate di volta in volta, quindi l'idea di mantenere un codice corretto ed ordinato e' alla base. Esempio:

```
1 <html>  
2   <head>  
3   </head>  
4   <body>  
5       Hello World!  
6   </body>  
7 </html>  
8  
9 <html><head></head> <body>  
10 Hello World!</body></html>
```

Annidamento

I tag HTML vengono annidati quindi abbiamo tag uno dentro l'altro. Uno degli aspetti fondamentali dei linguaggi di markup e' che sono fatti secondo la filosofia last in last out. I tag sono contenuti tra loro (o allo stesso livello), ma non si sovrappongono mai. Esempio:

```
1 <font color="blue">  
2 Hello  
3 <font color="red">  
4 World!  
5 </font>
```

In questo esempio notiamo che il browser colora in maniera successiva, prima colora tutto di blu e poi la parte di world diventa rosso.

2.5.4 Commenti

I commenti aumentano la leggibilita' del codice ed e' buona norma commentare solo le parti significative. Permettono insieme all'indentazione di orientarsi all'interno di un grosso documento. Esempio: <!-- questo e' un commento -->

2.5.5 Case

L'HTML e' un linguaggio case insensitive (non cambia se scriviamo le cose in maiuscolo o in minuscolo), tendenzialmente si scrive in minuscolo. Al contrario l'XHTML e' case sensitive ed obbliga ad usare il carattere minuscolo.

2.5.6 Altre caratteristiche

Non e' sensibile agli spazi: se mettiamo tanti spazi consecutivi il browser li considera uno spazio solo. La stessa cosa succede con le linee vuote, non le considera.

2.6 Testa (<head>)

Contiene informazioni/metadati relativi al documento, ovvero tutto cio' che non e' contenuto all'interno della visualizzazione della pagina, tranne il titolo che diventa il titolo della scheda del browser. In questa parte mettiamo anche del contenuto per la negoziazione delle capability, ad esempio l'header. I browser non presentano elementi che compaiono nella testa, ma rendono disponibili queste informazioni attraverso altri meccanismi.

2.6.1 elemento Title

Questo elemento, che e' presente solo nella testa, indica il titolo della pagina e viene visualizzato in alto a sinistra nei browser piu' datati oppure nella scheda nei browser piu' recenti.

2.6.2 Metadati

Elemento <meta> : Definisce informazioni relative alla pagina piuttosto che contenuto vero e proprio. Contiene tre attributi principali:

- Name : contiene il nome della proprieta' come ad esempio Author, Description, Copyright, Generator, Language, Keywords
- Content : contiene il valore della proprieta'
- http-equiv : specifica caratteristiche dell'HTTP response header

Esempi:

- <META NAME="author" CONTENT="Claudio">
- <META NAME="description" CONTENT="...">
- <META NAME="copyright" CONTENT="...">
- <META NAME="keyword" CONTENT="...">

Queste informazioni non vengono presentate nella pagina, ma sono informazioni di contesto che descrivono il contenuto della stessa.

Meta tag - Keywords

Indica e dà informazioni su cosa un utente si dovrebbe aspettare all'interno del sito, e sono delle vere e proprie chiavi di ricerca. Sono separati da virgola, punto e virgola o spazio. I vari motori di ricerca vanno a ricavare le varie chiavi di indicizzazione che ci portano al sito quando facciamo una ricerca. Dobbiamo evitare di ripetere le stesse parole, evitare termini generici, di ripetere le stesse parole inoltre dobbiamo ripetere il termine in italiano, inglese, singolare e plurale. A volte succede che chi fa pubblicità su determinati motori di ricerca paga per click.

2.6.3 Attributo HTTP-EQUIV

Contiene informazioni che guidano la comunicazione tra il server ed il browser. Da informazioni sulla comunicazione attraverso server-browser, può essere usato al posto dell'attributo name. Fornisce un header HTTP per informazioni/valore dell'attributo content. All'interno del tag meta appartiene all'header di una pagina web. Il valore content-type appartiene all'header della comunicazione HTTP. Attraverso l'attributo content possiamo simulare un HTTP response header. Alcuni valori sono:

- content-type
- default-style
- refresh

Esempi

Esempio 1 :

```
1 <meta http-equiv="Content-Type" content="text/html; charset=iso-
2 8859-1">
```

Esempio 2:

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//HTML 4.01
2 Transitional//IT">
3 <html>
4   <head>
5     <meta http-equiv="Content-Type"
6     content="text/html; charset=iso-8859-1">
7     <title>Pagina di prova</title>
8   </head>
9   <body>
10    <!-- Scriveremo qui -->
11    Hello World!
12  </body>
13 </html>
```

2.7 Corpo (<body>)

Il corpo di un documento contiene il contenuto vero e proprio. Il browser presenta il contenuto in diversi modi:

- Browser visuali: e' una tela su cui disegnare testi, immagini colori
- Agenti audio: lo stesso contenuto e' parlato

Contiene tutte le informazioni per la visualizzazione della pagina, siccome gli style sheet sono il modo suggerito per specificare come verra' presentato un documento gli attributi del BODY sono stati deprecati.

2.7.1 Elementi inline vs block

Alcuni elementi nel body sono "block-level" altri "inline". Gli elementi block-level possono contenere elementi inline e altri elementi block-level, gli elementi inline contengono solo dati e altri elementi inline. Sono formattati diversamente e gli elementi block-level iniziano su una linea nuova mentre gli elementi inline no.

2.7.2 Testo

Tag-contenitori di testo (block-level), con l'unica eccezione dello

- <h1>, <h2> ... (h = heading) sono titoli, i numeri indicano la grandezza del carattere, vanno da h1 ad h6, esempio : <h1>Titolo1</h1>
- <p> sono paragrafi, unita' di suddivisione del testo ovvero lascia una riga vuota prima e dopo il testo, esempio: <p>Paragrafo</p>.
- inline che permette di definire la struttura della pagina ma non hanno significato, come se andassimo a definire una sorta di annotazione standard che riutilizziamo volta per volta. E' un contenitore generico che puo' essere annidato.
- <div> block-level che mette il contenuto uno sotto l'altro, va a capo ma non lascia righe di spazio, esempio <div>blocco 1</div>. Sono fondamentali per andare a strutturare la pagina. E' un blocco contenitore testo che va a campo ma non lascia righe di spazio. Non impone rappresentazioni di default (a parte il fatto di essere un elemento block-level)

Andare ad identificare sezioni della pagina con paragrafi, div e' importante perche' poi possiamo dire e modificare il comportamento di ogni singolo blocco

Esempio di una pagina che contiene un titolo, un paragrafo, un blocco di testo ed un contenitore:

```
1 ...
2 <h2>Titolo della pagina</h2>
3 <p>paragrafo 1</p>
```

```

4 <p>paragrafo 2</p>
5 <div>blocco 1</div>
6 <div>blocco 2
7   <span>contenitore 1</span>
8   <span>contenitore 2</span>
9 </div>

```

Struttura della pagina con tag <div>

Il tag div e' uno dei tag piu' utilizzati nella creazione di pagine web, soprattutto per la creazione di layout perche' fornisce un vero e proprio elemento strutturale della pagine e suddivide gli spazi in zone per progettare il sito in modo semplice e dettagliato in sinergia con i fogli di stile. Facciamo un esempio immaginando di dover costruire un sito pensiamo a come debba essere strutturato, tipicamente abbiamo :

- Un contenitore (container)
- Una parte alta (header)
- Un corpo centrale (middle)
- Un menu' (navigation)
- Una parte bassa (footer)
- Spesso anche un pannello laterale (sidebar)

```

1 <div id="container">
2   <div id="header">
3     <div if="navigation"></div>!--#navigation-->
4   </div><!--#header-->
5   <div id="main"></div><!--#main-->
6   <div id="sidebar"></div><!--#sidebar-->
7   <div id="footer"></div><!--#footer-->
8 </div><!--#container-->

```

2.7.3 Stili

Esistono due tipi di stili:

- Fisici: forniscono lo stile grafico del testo
- Logici: forniscono informazioni sul ruolo svolto dal testo e non per forza impongono uno stile grafico

Stili fisici

Tutti elementi inline che cambiano come visualizzazione della pagina. Tutti devono essere chiusi dal loro tag.

- `<b>` bold: formatta testo in grassetto
- `<i>` italic: formatta il testo in corsivo
- `<u>` underline: sottolinea il testo
- `<strike>`: testo barrato
- `<sup>` apice
- `<sub>` pedice

Stili logici

Definiscono la semantica della pagina e possono avere stili fisici associati.

- `<abbr>` abbreviazione: non comporta nessun cambiamento grafico
- `<address>` indirizzo, `<cite>` citazioni: testo in corsivo
- `<samp>` esempio: testo a spaziatura fissa

Font (`<font>`)

Font = colore, dimensione e tipo del carattere. Anche in questo caso meno li utilizziamo nella struttura della pagina e meglio stiamo. Fare attenzione al colore per una questione di usabilit . Il carattere predefinito Times new roman   poco leggibile, meglio Verdana, Arial o Helvetica. Esempio:

```
1 <font face="verdana, arial, helvetica, sans serif">testo</font>
```

E' buona norma utilizzare caratteri scuri, non indicare un solo carattere. Oltre a questo possiamo scegliere il colore del testo e possiamo farlo in maniera indipendente oppure aggiungere attributi al tag font. Per scegliere il colore del testo:

```
1 <font color="#0000FF">testo blu</font>
2
3 <font color="#0000FF" face="verdana">testo
4 blu</font>
```

C'  la possibilit  di avere anche font annidati:

```
1 <font color="blue" face="verdana">
2     testo blu
3     <font color="red" >
4         testo rosso
5     </font>
6 </font>
```

Bisogna ricordarsi che riempire attributi alla pagina web non va bene, ci sono i fogli di stile. Possiamo anche modificare la dimensione, con l'attributo size. Abbiamo due modi per impostare la dimensione, utilizzare un valore tra 1 e 7 oppure utilizzare il numero, che aumenta la dimensione base del carattere del numero che abbiamo indicato.

2.7.4 Elenchi

- ordinati
- non ordinati
- elenchi di definizioni

sintassi:

```
1 <elenco>
2   <elemento> primo elemento
3   <elemento> secondo elemento
4 </elenco>
```

La chiusura dell'elemento e' opzionale.

Elenchi ordinati

C'e' la numerazione degli elementi ed e' progressiva. (ordered list) e' il tag per l'elenco ordinato, (list item) e' il tag per elemento, e' per la lista non ordinata. Esempio:

```
1 <div>Titolo</div>
2 <ol>
3   <li>Primo elem prima tabella
4   <li>Secondo elem prima tabella
5   <ol>
6     <li> Primo elem seconda tabella
7     <li> Secondo elem seconda tabella
8   </ol>
9 </ol>
```

Elenchi non ordinati

Esempio:

```
1 <ul>
2   <li> Primo elemento
3   <li> Secondo elemento
4   <ul>
5     <li> Primo elemento
6     <li> Secondo elemento
7   </ul>
8 </ul>
```

Elenchi di definizioni

- Elenco di definizione <dl> (definition list)
- Termine da definire <dt> (definition term)
- Definizione del termine <dd> (definition description)

Esempio:

```

1 <p> Titolo
2 <dl>
3   <dt>&lt;p>
4     <dd>apertura paragrafo
5   <dt>&lt;div>
6     <dd>apertura blocco di testo
7   <dt>&lt;span>
8     <dd>apertura elemento inline
9   Altri tag...
10 </dl>
```

2.8 Ipertestualita' (Link)

Il Link e' un ponte tra una pagina web ed un'altra, sia esternamente che internamente. Sono formati da contenuto (testo o immagine) e la risorsa puntata.

2.8.1 Link

Sono formati da due componenti:

- Il contenuto (testo o immagine) che nasconde il collegamento; il link e' come se fosse un puntatore
- La risorsa puntata

La sintassi: Clicca qui per visualizzare il collegamento. La testa del link e' "qui" mentre la coda e' "indirizzo". Un tipo particolare di link e' quello di link ad indirizzo mail. Per manipolare i link si usa CSS, anche se in realta' si puo' fare con HTML ma e' sconsigliato. Esempi di link:

- Link a pagina HTML:
 - Clicca qui
- Link a documenti:
 - Clicca qui
- Link ad immagini:
 - Clicca qui
- Link ad altri file
 - Clicca qui

Caratteristiche

Ci sono una molteplicità di stati del link:

- Link a riposo di colore blu <body alink
- Link visitato di colore violetto
- Link attivo
- Link al passaggio del mouse solo con fogli di stile (CSS)

E' possibile manipolare i link attraverso degli attributi, ma e' sconsigliato farlo, per manipolare i link si usano i fogli di stile (CSS)

Percorsi assoluti

Viene indicato il percorso per esteso.

```
1 <a href="http://www.miosito.it/cartella/miofile.html"> qui
2 </a>
```

Mentre nel caso di file presente sull'hard disk:

```
1 <a href="c:\cartella\miofile.html"> qui
2 </a>
```

Percorsi relativi

La pagina e' nella stessa directory di quella in cui siamo.

```
1● <a href="link.html">Clicca qui</a>
2
```

Indica al browser di cercare il file nella stessa directory

```
1● <a href="sottodir/sottolink.html">Clicca qui</a>
2
```

Indica al browser di cercare il file nella sottodirectory sottodirectory.

```
1● <a href="../superlink.html">Clicca qui</a>
2
```

.. Indica al browser di cercare il file nella directory padre

Considerazioni

Nei nomi dei file evitare gli spazi ed e' meglio non utilizzare - e ricordarsi che le maiuscole e minuscole fanno la differenza. Mettere il percorso web che serve al server per arrivare al file.

Link interni

Ancore sono i link che mi permettono di saltare da una sessione all'altra della pagina.

- Ancora `<a name="primo"> ancora </a>`
- Riferimento all'ancora `<a href="#primo"> vai all'ancora </a>`
- Riferimento ad inizio pagina `<a href="#"> torna su </a>`

2.8.2 Immagini

Il web non e' solo ipertesto ma bensì ipermedia. Per inserire un un immagine si ha un attributo `src`: sorgente del file ovvero il percorso di memorizzazione del file, un attributo `alt`: descrizione mostrata al passaggio sull'immagine e gli attributi `width` e `height`: dimensione dell'immagine.

```
1 
```

Immagine come link:

```
1 <a href="destinazione.html">
2 
3 </a>
```

2.8.3 Tabelle (table)

Hanno rappresentato per molto tempo una cosa molto importante per l'HTML, il loro ruolo e' andato oltre quello della semplice tabella, ma la usavano come strutturazione della pagina web. Definizione tramite elemento `<table>`, ogni riga definita con il tag `<tr>` contenuto in `table`, ogni cella definita con il tag `<td>` contenuto in `<tr>`. Una riga puoi' essera anche divisa in `table heading` usando l'elemento `<th>` e l'elemento `<caption>` che va a definire l'elemento della tabella. Un esempio di tabella:

```
1 <table style="width:100%">
2 <tr>
3 <td>Jill</td>
4 <td>Smith</td>
5 <td>50</td>
6 </tr>
7 <tr>
8 <td>Eve</td>
9 <td>Jackson</td>
10 <td>94</td>
11 </tr>
12 <tr>
13 <td>John</td>
14 <td>Doe</td>
15 <td>80</td>
16 </tr>
17 </table>
```

Attributo border se non e' specificato non e' presente il bordo, altrimenti
`<table border="1" style="width:100percento">`

2.8.4 Form <form>

Utilizzate per collezionare input dell'utente, specifica diversi elementi block-level come input, checkbox, radio button, submit.

Text input

`<input type="text">` definisce un campo testo di una linea, `<input type="password">` e' un tipo speciale di campo testo dove il valore inserito viene nascosto.

```
1 <form>
2 First name:<br>
3 <input type="text" name="firstname">
4 <br>
5 Last name:<br>
6 <input type="text" name="lastname">
7 </form>
```

Text area

Definisce un campo testo a linee multiple.

```
1 <textarea name="message" rows="10" cols="30">
2 The cat was playing in the garden.
3 </textarea>
```

Radio button input

`<input type="radio">`, definisce un radio button permette una scelta tra diverse opzioni.

```
1 <form>
2 <input type="radio" name="sex" value="male"
3 checked>Male
4 <br>
5 <input type="radio" name="sex" value="female">Female
6 </form>
```

Checkbox input

`<input type="checkbox">`, Definisce una serie di checkbox che permettono una scelta multipla tra diverse opzioni.

```
1 <form>
2 <input type="checkbox" name="vehicle1" value="Bike"> I
3 have a bike
4 <br>
5 <input type="checkbox" name="vehicle2" value="Car"> I
6 have a car
7 </form>
```

Drop-down list

`<select>`, definisce un menu' a tendina. L'elemento `<option>` definisce possibili opzioni, `<option value="fiat" selected>Fiat</option>`.

```
1 <select name="cars">
2 <option value="volvo">Volvo</option>
3 <option value="saab">Saab</option>
4 <option value="fiat">Fiat</option>
5 <option value="audi">Audi</option>
6 </select>
```

Submit button

`<input type="submit">`, definisce un bottone per sottomettere la form ad un handler. Un Handler e' una pagina lato server che processa i dati della form che e' definito con l'attributo `action`.

```
1 <form action="action_page.php">
2 First name:<br>
3 <input type="text" name="firstname" value="Mickey">
4 <br>
5 Last name:<br>
6 <input type="text" name="lastname" value="Mouse">
7 <br><br>
8 <input type="submit" value="Submit">
9 </form>
```

Action attribute

Definisce le azioni che devono essere fatte quando la form e' inviata, la sottomissione e' fatta solitamente con il bottone `submit`.

Method attribute

Specifica l'opzione HTTP (GET o POST) da usare per inviare la form.

- `<form action="action.page.php" method="get">`
- `<form action="action.page.php" method="post">`

GET e' usata di default per invio di form passive e senza informazioni sensibili, perche' i dati inviati tramite la form sono visibili nell'indirizzo ed e' ottima per piccole quantita' di dati. Al contrario POST fornisce maggiore sicurezza, poiche' i dati non sono visibili nell'indirizzo.

Name attribute

Ogni campo `input` deve avere un nome.

```

1 <form action="action_page.php">
2 First name:<br>
3 <input type="text" value="Mickey">
4 <br>
5 Last name:<br>
6 <input type="text" name="lastname" value="Mouse">
7 <br><br>
8 <input type="submit" value="Submit">
9 </form>

```

Fildset

Raggruppa campi in una form tramite elemento <fieldset>, si puo' definire una caption per la form <legend>.

```

1 <form action="action_page.php">
2 <fieldset>
3 <legend>Personal information:</legend>
4 First name:<br>
5 <input type="text" name="firstname" value="Mickey">https://it.
   overleaf.com/project/603f5235f1d064dc7e504cb7
6 <br>
7 Last name:<br>
8 <input type="text" name="lastname" value="Mouse">
9 <br><br>
10 <input type="submit" value="Submit">
11 </fieldset>
12 </form>

```

Altri attributi

- Attributo value specifica il valore iniziale:
 - <input type="text" name="firstname" value="John">
- Attributo readonly specifica che il valore non puo' essere cambiato:
 - <input type="text" name="firstname" value="John" readonly>
- Attributo disabled specifica un campo disabilitato e non utilizzabile (non viene inviato):
 - <input type="text" name="firstname" value="John" disabled>
- Attributo size specifica la dimensione in caratteri del campo:
 - <input type="text" name="firstname" value="John" size="40">
- Attributo maxlength specifica la dimensione massima del campo:
 - <input type="text" name="firstname" maxlength="10">

2.9 XHTML

Significa esplicitamente "EXtensible HyperText Markup Language", e' quasi identico all'Html ma ha delle regole piu' stringenti.

2.9.1 Convertire HTML in XHTML

- Aggiungere un XHTML `<!DOCTYPE>` nella prima linea di ogni pagina
- Aggiungere un attributo `xmlns` all'elemento `html` di ogni pagina
- Cambiare tutti i nomi di elemento in minuscolo
- Chiudere tutti gli elementi vuoti
- Cambiare tutti i nomi di attributo in minuscolo
- Quotare tutti i valori di attributi

Chapter 3

CSS

CSS

3.1 Introduzione

CSS sta per fogli di stile a cascata ed e' uno dei linguaggi fondamentali del W3C. E' stato creato per separare la struttura del sito, fatta con HTML, alla formattazione dello stesso. Questa separazione ci permette di cambiare la formattazione del sito molto piu' agilmente, quindi c'e' anche una riduzione del margine di errore.

3.1.1 A cosa servono

Servono ad oltrepassare i limiti dell'HTML associando uno stile al testo delle pagine, non solo colori o font ma anche le animazioni. E' un linguaggio che descrive la presentazione di un documento HTML, ovvero descrive come gli elementi devono essere mostrati, stampati o su altri media, ad esempio, imposta stili diversi per titoli e paragrafi, sfrutta i benefici dell'indentatura e della giustificazione. Grazie a questo abbiamo una gestione del sito facilitata perche' riducono il carico di lavoro, permettono di controllare l layout di pagine web multiple in un colpo solo. I CSS possono essere separati dal documento. Il risultato sono pagine piu' leggere e facili da modificare, sono uno strumento portentoso per l'accessibilita', anche grazie al fatto di poter essere gestiti con linguaggi di scripting avanzati in grado di modificare con un solo click l'aspetto di una pagina.

3.2 Meccanismi, costrutti e selettori

3.2.1 com'e' fatta una regola

Un foglio di stile e' un insieme di regole:

- h1: selettore definisce quale parte del sito dev'essere gestito secondo le dichiarazioni nel blocco delle dichiarazioni
- ... blocco delle dichiarazioni coppie proprieta', valore: definiscono un aspetto dell'elemento da modificare (margini, colore di sfondo, ...) secondo il valore espresso

```
1 h1 {color : white; background: red}
```

- Dichiarazione valida singola con valori multipli:

```
1- p {font: 12px Verdana, Arial;}
2
```

- E' possibile fare piu' dichiarazioni separate da ;

```
1- p {color: red; text-align: center;}
2
```

- Esempio errato:

```
1- body {color background: black;}
2
```

Proprieta' singole e a sintassi abbreviata

Per ogni elemento e' possibile definire una sintassi singola:

```
1 div { margin-top: 10px; margin-right: 5px; margin-bottom: 10px;
margin-left: 5px;}
```

e un abbreviata:

```
1 div {margin: 10px 5px 10px 5px;}
```

Questo esempio imposta i margini di un elemento contenitore div.

3.2.2 Selettori

Selettore di elementi (tyoe selector), e' costituito da uno qualunque degli elementi di HTML, spesso riferito come tag selector invece che type selector. Sintassi:

```
1 h1 {color: #000000;}
2 p {background: white, font: 12px Verdana, Arial, sans-serif}
3 table {width: 200px;}
```

Combinazioni

Il CSS e' efficace piu' siamo specifici, al contempo se diventiamo troppo specifici il browser ha maggior latenza per visualizzare la pagina ed il programmatore ha piu' difficolta' a capire quello che e' stato fatto.

3.2.3 Selettore (attributi)

Servono per selezionare un elemento sulla base di un valore.

attribute : seleziona tutti gli elementi con attributo attribute

3.2.4 Raggruppare

E' possibile raggruppare diversi elementi, elementi separati da virgola:

```
1 h1, h2, h3 {background: white;}
```

Se riusciamo a raggruppare evitando di ridurre troppo la leggibilita' risparmiamo byte.

3.2.5 Inserire i fogli di stile in un documento

- Esterni: e' un foglio di stile separato dal documento e poi mettiamo link all'inizio, utilizzo dell'elemento `<link>`, inserendo il tag `<link>` all'interno dell' `<head>`
- Interno: e' un foglio di stile il cui codice e' compreso in quello del documento HTML
- In linea: quando lo stile e' interno al tag HTML target

Rispetto a queste diverse modalita' si parla di fogli di stile collegati, incorporati, in linea.

CSS interni

Inseriti direttamente l'elemento `<STYLE>` all'interno della testa `<HEAD>`

```

1  ...<head>
2  <style type="text/css">
3  body {
4  background: #FFFFCC;
5  }
6  </style>
7  </head>...
```

Attributi type(obbligatorio), media(opzionale)

CSS in linea

Attributo style, la dichiarazione avviene a livello dei singoli tag contenuti nella pagina (fogli di stile in linea). Sintassi:

```

1  <elemento style="regole_di_stile">
```

Questi stili sono molto potenti ma poco usati, perche' devo cambiarli in ogni file e tag, sovrascrivono sia quelli interni che esterni. Esempio:

```

1  <h1 style="color:purple">Testo</h1>
```

3.2.6 Selettori speciali

Voglio essere anche in grado di dire che ci sono div di una certa classe Class e Id, i due selettori devono essere associati ad una stylesheet altrimenti perdono di significato.

Selettore class

Selettori che non mappano direttamente a uno specifico tag, ad esempio cambiare il colore di parola all'interno di un tag paragrafo. In questo caso si usano i selettori class, permettono di scegliere liberamente il nome del selettore, meglio se il nome descrive anche il significato (evidenziarosso, colorarosso), nome preceduto da "."

Pseudo classi

Applicare uno stile al verificarsi di un determinato evento. `:active` e' usato per selezionare e impostare lo stile di elementi «attivi»

Selettore id

Identificano lo stile di un singolo elemento all'interno della pagina HTML, esatto quando si e' sicuri che quell'elemento comparira' una sola volta. Come per i selettori class permetotno di scegliere liberamente il nome del selettore, sempre meglio se il nome descrive anche il significato. Nome preceduto da `#`. L'id dev'essere univoco, perche' useremo funzioni di javascript che richiamano un id. Quando un elemento ha un senso che sia unico bisogna stare attenti.

3.3 Ereditarieta'

Un elemento contenuto in un altro (child) eredita tutte le proprieta' di stile dal suo elemento padre, tuttavia, se il figlio (o discendente) ha proprieta' in conflitto con le proprieta' definite dall'elemento padre, il conflitto viene risolto in favore dall'elemento figlio, la proprieta' piu' specifica vince.

- Ereditarieta' esplicita: associare a una proprieta' il valore `inherit` per indicare che la proprieta' ereditera' il valore dell'elemento padre.

3.4 Regole CSS: valutazione a cascata

Uno User agent deve applicare un algoritmo per stabilire i valori di combinazione di elementi e proprieta'. Per prima cosa bisogna selezionare tutte le dichiarazioni che applicano a un elemento/proprieta' per il media type richiesto.

- Il primo ordinamento e' fatto in bsae a peso e origine:
-

3.5 Font e web

Nel design di pagine web si puo' formattare testo in maniera simile alle applicazioni di word processing. Si ha la necessita' di specificare un font che e' dispobibile nel browser lato utente, altrimenti si hanno problemi di visualizzazione, in caso di problemi il browser sostituisce il font con un'altro. Il tag `font-family` definisce il font da usare:

```
1 body {
2   font-family: "Trebuchet MS", Thaoma, Arial, sans-serif;
3   font-size: small;
4 }
```

Font-size definisce la dimensione del testo, ci sono diverse opzioni di definizione:

- **Absolute-size:** un insieme di keyword definiscono le dimensioni predefinite: xx-small, x-small, small, medium, large, x-large, xx-large
- **Length:** un numero seguito da una unita' di misura assoluta (cm, mm, in, pt, pc) o un numero seguito da un unita' di misura relativa (em(relativa al font di default), ex, px), i pixel sembrano perfetti perche' sono l'unita' di misura dei monitor
- **Percentage:** un intero seguito da un %. Il valore e' la percentuale della dimensione del font dell'oggetto padre. Definisce la dimensione come la dimensione di default del browser.
- **Relative size:** un insieme di keyword interpretate come relative alla dimensione del font dell'oggetto padre. Possibili valori includono larger e smaller (rispetto al carattere di default, grosso modo nascondono una percentuale).

E' importante quando facciamo siti web responsive, perche' si adatta la dimensione in base alla dimensione dello schermo che stiamo usando. Proprieta' aggiuntive:

- font-style
- font-weight: quanto "pesante" e' il carattere
- font-variant

3.6 CSS box model

(Domanda che spesso propone). Tutti gli elementi di una pagina HTML sono modellati come delle box di cui noi percepiamo il content. In realta' intorno a quel contenuto abbiamo il padding, border e margin. Il padding e' un'area libera attorno al content (e' trasparente), il Border e' un bordo che circonda padding e content. Il margin e' un area libera attorno al bordo (trasparente). In questo modo abbiamo un modo per manipolare il posizionamento dell'oggetto in base a quello che gli sta intorno.

3.6.1 Border

Border-style per modificare il border: None, dashed, dotted, solid, double, groove, ridge, inset, outset. Esiste anche il border-top,bottom,left,right-style. Border-width e' definita in pexel o che le keyword thin, medium o thick. Border-color e' definita con nome (red, transparent), RGB (rgb(255,0)) o hex (#ff0000). Il bordo esiste sempre e di default e' none. Border e' un attributo scorciatoio, posso definire direttamente width, style e color:

```

1  p {
2      border:5px solid red;
3  }
```

3.6.2 Margin e Padding

Definiscono lo spazio tra gli elementi ed hanno gli stessi attributi di border. Funzionano allo stesso modo.

3.6.3 Line-height

Il testo e' mostrato usando line box. Andiamo a modificare le priorita' del testo tramite line-height.

3.7 Flow Processing

In normal flow processing, ogni elemento ha una box corrispondente. L'elemento HTML si riferisce a una box chiamata initial containing block (corrisponde all'intero documento). Il Box degli elementi figli sono contenuti in box degli elementi padri :

- Elementi block: fratelli sono messi uno sopra l'altro
- Elementi inline: fratelli sono messi uno di fianco all'altro

Cambiare il flow processing significa cambiare il modo in cui gli oggetti si relazionano con gli altri.

```

1  html, body {border: solid red thin}
2  html {border-width: thick}
3  body {padding: 15px}
4  div {margin: 0px; padding: 15px; border: solid black 2px}
5  .shade {background-color: green}
6  .topMargin {margin-top: 10px}
7
8
9  <body>
10   <div id="d1">
11     <div id="d2">
12       <div id="d3" class="shade"></div id="d3">
13     </div id="d2">
14     <div id="d4" class="topMargin"></div id="d4">
15   </div id="d1">
16 </body>

```

Gli elementi block compaiono in ordine rispetto all'ordinamento nel documento HTML.

3.7.1 Display

La proprieta' display controlla il layout e dice se e come un elemento deve essere mostrato. Il valore di default block o inline. Esiste anche il valore none, che non ci mostra l'oggetto nella pagina.

3.7.2 Block

Un elemento block e'ha proprieta' display e valore block inizia sempre su una riga nuova. Lo style sheet dello user agent (non e' CSS) specifica valori de default. Elementi block tradizionali includono html, body, p, pre, div, form, ol, ul, dl, hr, h1-h6. Molti degli altri elementi accettano li e quelli relitve alle tabelle hanno la proprieta' display con valore inline. Quando gli elementi block sono impilati, i margini adiacenti sono collassati nel margine piu' grande. Il valore iniziale della proprieta' width e' auto, gli elementi block occupano l'aera di contenuto piu' larga possibile, tutto fatto rispettando margini e padding. E' possibile specificare la lunghezza usando CSS width (di default il margine destro viene adattato per gestire il cambiamento della larghezza).

3.7.3 Inline

Box che corrispondono a celle di caratteri o elementi inline (display: inline) sono messe uno in fianco alle altre in line box. Elementi inline iniziano sempre sulla stessa riga e prendono la larghezza necessaria. Line box sono messe uno sopra le altre. Le altezza, a differenza degli elementi block, sono basate sul contenuto, le celle di caratteri sono allineati dalla baseline.

```
1 <span style="font-size: 36pt">BIG</span>
```

Il Padding, berder e margin influenzano la larghezza ma non l'altezza delle boc inline.

```
1 <div id="d3" class="shade">
2   Inserimento <span style="border: dotted
3     silver 10px">di</span> una linea di testo!
4 </div>
```

Per posizionare elementi inline all'interno delle line box si usa la proprieta' vertical-align: ci permette di allineare in verticale tutto cio' che abbiamo all'interno della box-line.

Il valore di default di un elemento puo' essere sovrascritto, ad esempio definito come inline per ottenere un menu' orizzontale:

```
1 li {
2   display: inline;
3 }
```

3.8 Oltre il normal flow

Se vogliamo andare oltre il flusso normale dobbiamo andare a dire come gli oggetti si relazionano tra di loro, indipendentemente dalla spaziatura.

3.8.1 Proprieta' position

Specifica il tipo di posizionamento:

- Static: e' di default. Indicazione esplicita al browser che stiamo utilizzando il normal-flow.
- Relative: il posizionamento e' relativo alla posizione tradizionale (quella con position:static), nessun elemento verra' posizionato per riempire il gap lasciato.
- Fixed: l'elemento posizionato nello stesso punto anche quando la pagina viene scorsa. Proprieta' top, bottom, left, right usate per posizionare l'elemento. (sito trenitalia lo stato della prenotazione rimane fisso)
- Absolute: il posizionamento box relativo alla posizione dell'elemento antenato (quello con position:relative). Se non esistono antenati si considera il body e l'elemento mantiene la posizione quando la pagina viene scorsa

Absolute positioning:

```

1  p {
2      position: relative;
3      margin-left: 10em;
4      width: 8em;
5  }
6  .marginNote {
7      position: absolute;
8      top: 0;
9      left: -10em;
10     width: 8em;
11     background-color: yellow;
12 }
13 <p>
14     Questo e' il corpo della nota <span class="marginNote">Questa
15     invece e' la nota a
16 </p>

```

In pratica in questo esempio vediamo come mettere delle note a margine, il corpo sta tutto a destra e le note vanno a sinistra.

3.8.2 Sovrapposizione degli elementi

z-index specifica la gestione di elementi che si sovrappongono:

```

1  #text {
2      position: absolute;
3      top: 10px; left: 10px;
4      background-color: yellow;
5      letter-spacing: 0.1ex;
6      z-index: 1;
7  }
8  #overlay {
9      position: absolute;
10     top: 10px; left: 10px;
11     width: 1.1ex; height: 5em;
12     border: solid red 1px;
13     z-index: 2;
14 }

```

3.8.3 Proprieta' float

Permette di posizionare un testo intorno ad un'immagine : text wrap o rounaround. CSS ottiene questo effetto permettendo a elementi che seguono un elemento float in HTML di circondare l'elemento, cambiando la lora posizione. Float permette anche di creare pagine a colonne : assume valore left, right, none. Quindi spesso possono esserci errori di visualizzazione se non si sta attenti. Una soluzione e' quella di forzare i due elementi parenti uno a sinistra e l'altro a destra.

3.8.4 Proprieta' clear

Se si vuole forzare un elemento a stare da solo senza elementi a lato si usa la proprieta' clear. Specifica su quale lato di un elemnto non possono essere visualizzati elementi float. Si possono evitare elementi:

- A sinistra clear: left
- A destra clear: right
- A sinistra e destra clear: both

3.8.5 Esempio struttra

Ritorniamo alla struttra della pagina gia' vista in precedenza:

```

1 <div id="container">
2   <div id="header">
3     <div id="navigation"></div><!--#navigation-->
4   </div><!--#header-->
5   <div id="sidebar"></div><!--#sidebar-->
6   <div id="main"></div><!--#main-->
7   <div id="footer"></div><!--#footer-->
8 </div><!--#container-->
```

Ora usiamo le proprieta' float e clear per posizionare il main e la sidebar sulla stessa linea e il footer sotto di loro.

```

1 #main {
2   width:700px
3   height: 300px;
4   background-color: #47834B;
5   margin: 10px;
6   float: left;
7 }
8
9 #sidebar {
10  width: 300px;
11  height: 300px;
12  float: right;
13  background-color: #72C29B;
14  margin: 0px 10px;
15 }
16
17 #footer {
```

```
18     clear: both;
19     width: 1000px;
20     height: 70px;
21     background-color: #C4C4C4;
22     padding: 10px;
23     margin: 10px;
24 }
25
26 #container{
27     width:1040px;
28     margin:0 auto;
29 }
```

Tutta la parte della visualizzazione della pagina con le tabelle le lascia solo come cenno storico, non si fa piu' cosi' perche' e' un disastro riadattare e modificare le pagine scritte cosi'.

Chapter 4

JavaScript

4.1 Introduzione

JavaScript non ha niente a che fare con Java, non confondiamo il linguaggio di programmazione Java con JavaScript che e' un linguaggio di scripting. La pagina statica e' solo HTML+CSS, una pagine dinamica il cui contenuto viene generato al momento della richiesta HTTP.

4.1.1 Applicazione Web

E' un software sviluppato ed utilizzato attraverso tecnologie web:

- Linguaggi mark-up, scripting, programmazione
- Tecnologie client-side e server-side

4.1.2 Linguaggi

Linguaggi di mark-up descrivono documenti strutturati (contenuto + struttura), abbiamo gia' visto HTML. I linguaggi di programmazione descrivono programmi come sequenza di istruzioni. I linguaggi di scripting, sono un tipo particolare di linguaggio di programmazione (ad es. PHP, JavaScript), per scrivere script, tendenzialmente lo script e' un piccolo programma che inseriamo nella pagina web.

Linguaggi di scripting

Il programma e' un insieme di istruzioni, i linguaggi di programmazione sono linguaggi per scrivere programmi. I linguaggi sono composti da:

- Alfabeto: insieme di simboli con cui possiamo costruire i termini
- Sintassi: grammatica che fornisce le regole di composizione dei termini
- Semantica: significato delle frasi ben formate

Il parser analizza frasi e decide se sono frasi ben formate del linguaggio o no.

Il programmatore scrive il programma sorgente utilizzando un linguaggio ad alto livello che poi viene tradotto in linguaggio macchina.

4.1.3 Livelli logici

Si sviluppano su tre livelli logici:

- Presentazione: HTML, CSS
- Intermedio - JavaServlet, JSP, PHP, ASP, JavaScript, Flash
- Dati - RDBMS, XML, JSON

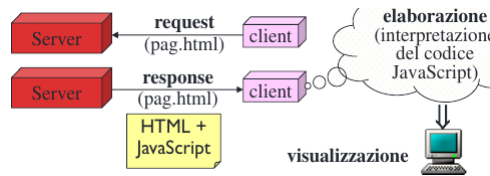


Figure 4.1: Clie Side

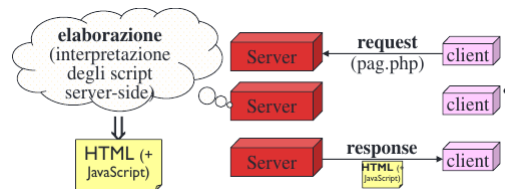


Figure 4.2: Server Side

4.2 JavaScript

A differenza di HTML, JavaScript e' case-sensitive, purtroppo possono esserci incompatibilita' e differenze tra i diversi browser, a volte si comportano in maniera diversa o non funzionano. Si basa su due concetti principali:

- DOM (Document Object Model): ci permette di modificare e puntare i diversi elementi della pagina.
- Eventi (script event-driven): ad esempio on mouse over, vengono catturati dagli event endler e permettono di modificare il comportamento della pagina a seconda del comportamento che un utente ha interagendo con la pagina stessa.

4.2.1 Tipi

JS e' un linguaggio debolmente tipato, il tipo delle variabili (e dei parametri/argomenti delle funzioni) non viene dichiarato esplicitamente, ma definito implicitamente al primo assegnamento: `numero = 1`, `numero` e' di tipo `Number`, si puo' riassegnare il tipo di `numero`. JS converte automaticamente i tipi durante l'esecuzione (quando possibile). Tipi principali:

- Number: interi e decimali (virgola mobile)
- Boolean: valori booleani
- String: sequenze di caratteri

4.2.2 Variabili ed istruzioni

Dichiarazione variabili:

- Non e' obbligatorio l'uso della keyword var
- Sono senza tipo
- var variabile;

Dichiarazione delle variabili locali:

- E' obbligatorio l'uso della keyword var, se invece non mettiamo il var viene interpretata come variabile globale
- Sono senza tipo
- var prezzo_scontato;

Le inizializzazioni sono facoltative (ma e' buona norma farle all'atto della definizione e prima di utilizzare le variabili). Tutte le istruzioni JS devono terminare con punto e virgola. Loosely typed, possono assumere diversi tipi a seconda del momento:

- alfa = 10
- beta = "Claudio"
- alfa = "Erica" // tipo diverso

Sono consentiti incrementi, decrementi e operatori di assegnamento estesi (++ , - ..). Hanno due possibili scope:

- Globale, per le variabili definite fuori da funzioni
- Locale, per le variabili definite esplicitamente dentro a funzioni (compresi i parametri ricevuti)

Attenzione che un blocco non delimita uno scope! tutte le variabili definite fuori da funzioni, anche se dentro a blocchi innestati sono globali:

```
1 x = '3' + 2; {  
2   {x = 5};  
3   y = x + 3;  
4 }
```

Operatore typeof ritorna il tipo di una espressione. Il tipo e' dinamico: rappresenta il tipo in quel momento temporale e corrispondente al valore attuale della variabile (o dell'oggetto...)

4.2.3 Costanti

Le costanti numeriche sono sequenze di caratteri numerici non racchiuse da virgolette o apici, tipo number. Le costanti booleane sono true e false, altre costanti sono null, Nan e undefined. I commenti sono come in Java.

4.2.4 Operatori

Abbiamo gli operatori classici, ricordiamo che `=` e' l'assegnamento, mentre `==` e' il confronto. `===` e' un operatore di confronto che non solo controlla se `A == B` ma controlla anche il tipo.

4.2.5 Espressioni

Sono simili a quelle java:

- Espressioni numeriche: somma, sottrazione, prodotto, divisione, modulo, shift ...
- Espressioni condizionali con: condizione ? assegnamento se condizione = true : assegnamento se condizione = false
- Espressioni stringa: concatenazione con `+`
- Espressioni di assegnamento con `=`
- `document.write(a/b)` : stampa nella pagina HTML il contenuto della parentesi

4.2.6 Condizioni booleane

Una condizione booleana e' un'espressione che ha valore vero (true) o false (false). Una condizine classico e' nel login, se la condizione che mi hai passato e' true allora diventa riconosciuto, altrimenti non lo riconosco.

4.2.7 Stringhe

Delimitate sia da virgolette sia da apici singoli. Per annidare virgolette e apici, occorre alternarli:

- `documenti.write('IMG src="image.gif"')` stampa nella pagina un tag HTML, che verra' poi interpretato dall'interprete HTML.

La concatenazione con `+`. Le stringhe JS sono oggetti dotati di proprieta' (ad esempio `length`) e metodi (ad es. `substring(first, last)`)

4.3 Specifica dello script

Il codice di un programma JS viene incluso in un file HTML per mezzo del tag `<SCRIPT>`:

- Sono possibili piu' programmi nella stessa pagina
- Una pagina HTML puo' contenere piu' tag `<script>`

```
1 <BODY>
2   <SCRIPT language="JavaScript">
3       prima istruzione;
4       seconda istruzione;
5       terza istrzione;
6   </SCRIPT>
7 </BODY>
```

L'interprete HTML lo invia all'interprete JS. Se andiamo ad inserire uno script come sequenza di istruzioni quelle vengono eseguite in quell'esatta posizione senza chiamate.

4.3.1 Definizione funzione

Le definizioni delle funzioni vengono generalmente incluse nella sezione HEAD. I richiami alle funzioni (predefinite nel linguaggio o definite dal programmatore) avvengono dove occorre, nel body della pagina HTML.

4.4 Document Object Model (DOM)

E' uno standard W3C. Definisco uno standard per accedere ai documenti. Ci permette di andare a modificare in generale il contenuto e la definizione di una pagina. E' separato in tre parti:

- Core DOM: modello standard per tutti i tipi di documenti
- XML DOM: modello standard per documenti XML
- HTML DOM: modello standard per documenti HTML

4.4.1 HTML DOM

Definisce:

- Elementi HTML come oggetti
- Proprieta' degli elementi HTML
- I metodi per accedere agli elementi
- Gli eventi per tutti gli elementi HTML

In altre parole e' uno standard che definisce come ottenere, cambiare, aggiungere o modificare elementi HTML. E' diviso in parte CORE e parte DOM. Ogni browser ha il suo DOM. E' organizzato secondo una gerarchia ad esempio come liste di oggetti.

4.4.2 Window

E' la finestra corrente del browser da cui poi partono tutti i figli. Ci da' molti metodi, ad esempio `window.alert(messaggio)`, propone una finestra di avviso con messaggio. E' usabile anche in un link e non restituisce nessun valore

4.5 Eventi form

Possiamo selezionare/deselezionare le checkbox al click su un link/bottone:

```
1 <FORM NAME="modulo">  
2   Opzione 1:<INPUT TYPE="checkbox" ID="box1" VALUE="v1">  
3 </FORM>  
4 <A REF
```

Con Javascript possiamo andare a personalizzare voci di menu', intercettare il click del mouse sul pulsante di invio di un form e di conseguenza mostrare un alert.

Chapter 5

HTML5

E' diventato di perse' qualcosa di piu' di markup, si identifica con la parola HTML5 una famiglia di tecnologie (HTML5 + CSS3 + JS). Viene utilizzato ed integrato anche nei sistemi operativi windows ed e' alla base delle app.

5.1 Limitazioni di HTML

Nasce il bisogno di andare oltre perche' spesso per animare una pagina abbiamo bisogno di librerie esterne (flash o quicktime) per eseguire audio/video. HTML non puo' memorizzare dati lato client se non con linguaggi di scripting o altre tecnologie. HTML non ha un formato nativo per il disegno: grafica o animazioni fornite come immagini o con Flash, Java. Html limita l'evoluzione del web.

5.2 HTML5

E' uno standard per strutturare e presentare il contenuto del World Wide Web. Aggiungiamo nuove funzionalita', nuovi tag e nuove API (per vedere il web come una sorta di servizio). La family e' composta da HTML5 + CSS3 + JS. Il motivo per cui viene introdotta questa famiglia e' per catturare l'evoluzione nell'utilizzo del web.

Una delle grandi differenze e' l'aggiunta di semantica, c'e' una migliore definizione delle componenti delle pagine web. I tag HTML forniscono informazioni aggiuntive utili per i motori di ricerca. Si decide di dare una migliore definizione delle componenti con tag HTML che sono autodocumentanti. Tutto porta a maggior comprensione ed inoltre alleggeriamo anche il CSS, perche' non abbiamo piu' bisogno di definire tanti id ma useremo direttamente il tag (header) come selettore. Fornisce supporto per tutto cio' che e' JS. E' pragmatico, parte dalla considerazione che i browser decidono cosa implementare e cosa no, non ha senso tradurre in specifica qualcosa che non viene utilizzato.

Lista di funzionalita':

- Nuovi elementi strutturali (header, footer, nav, div)
- Form 2.0 e validazione client-side
- Supporto nativo dei browser per audio e video (audio, video)
- Canvas API, idea di non inserire immagini embedded con tag image ad una definizione delle immagini che e' nativa
- Web storage: e' un pezzettino di memoria all'interno del browser e permette di memorizzare i dati come coppie chiave-valore, nel progetto diventera' il modo di sostituire il database.
- Offline application: possibilita' di lavorare offline anche grazie a web storage locale.
- Geolocation: identificare la posizione di un utente attraverso le web API

- Drag & Drop
- Web Workers: idea di avere dei thread che lavorano in background
- Nuove API di comunicazione (Server Sent Events, Web Sockets, ...)

Esempio:

```
1 <header> This is my header </header>
2
3 header {
4     width:960px;
5 }
```

Esempio di definizione di tag video e audio senza necessita' di plugin lato browser:

```
1 <video src="cat.mp4" width="400" height"300"> </video>
2 <audio src="big_seas_rip.mp3" controls preload="auto" autobuffer>
   </audio>
```

5.3 Concetti base

5.3.1 Sintassi

La sintassi fa un passo indietro rispetto ad XHTML, si torna al best effort. E' case insensitive, ovvero le maiuscole o minuscole non interferiscono con la validazione della pagina. I tag di chiusura e le virgolette non sono necessari.

5.3.2 Struttura della pagina web

HTML5 introduce 28 nuovi elementi. Definizione e scelta dell'elemento migliore per l'oggetto che si vuole rappresentare, non dobbiamo scegliere l'id migliore ma scegliamo l'elemento migliore che rappresenta l'obiettivo che abbiamo scelto. Questo da un'idea migliore di quello che stiamo facendo sia alla macchina che all'uomo. Hanno inoltre buttato via molte cose dell'intestazione.

Categorie di contenuti aggiunti:

- Metadati
- Flow
- Sectioning
- Heading
- Phrasing
- Embedded
- Interactive

5.3.3 Metadata

Configura la modalita' di presentazione e il comportamento del contenuto della pagina. Meta include parole chiave o descrizione della pagina.

5.3.4 Link

Rimuove tutto il contenuto inutile:

```
1 <link rel="stylesheet" href="foglio.css"></link>
```

lastessa cosa vale per gli script:

```
1 <script src="scriptfile.js"></script>
```

5.3.5 Flow

Modella il vero contenuto della pagina, contiene tutti gli elementi usati per marcare il contenuto, tradizionalmente contiene testo o file embedded.

5.3.6 Sectioning

Include quattro elementi: article, aside, nav e section. Definisce una struttura naturale del documento.

5.4 HTML5 Web Storage

Supporta memorizzazione dei dati ottenuti da internet nella propria macchina desktop.

- Application cache: salva la logica applicativa e l'interfaccia utente
- Offline storage: cattura dati generati dall'utente e risorse di interesse per l'utente

Ci concentreremo sull'offline storage, due semplici coppie chiave/valore, a file, a interi database SQL.

Dobbiamo stare attenti al furto di dati, sono connessi alla loro origine, possono essere acceduti solo dal dominio.

Il web storage API definisce uno standard (get, set) per andare a salvare o recuperare dati localmente su un computer utente o su un device. Prima si faceva tramite cookie. Abbiamo fino a 5Mb di dati creati dal sito web. E' importante anche per lo sviluppo offline di web application.

5.4.1 Effline storage

Dobbiamo decidere quale storage utilizzare:

- Session storage
- Local storage

Dobbiamo stare attenti perche' cambia la modalita' di fruizione del servizio, avere il login nel local storage mi permette di chiudere ed aprire l'applicazione e rimanere loggato, il session invece e' temporaneo, si cancella tutto quando chiudo e riapro

Il session storage permette di memorizzare dati specifici a una finestra, se l'utente visita lo stesso sito in due finestre diverse, ogni finestra avra' il suo session storage. Non e' persistente quando chiudiamo la finestra.

Il local storage permette di salvare dati persistenti sul computer dell'utente attraverso il browser. Quando un utente rivisita il sito ogni dato salvato nel local storage puo' essere recuperato.

I dati sono memorizzati come coppie chiave-valore. Sono simili ai cookie ma rimangono sempre al client non vengono mai spediti al server.

5.4.2 API

Rappresentano la modalita' di accesso alla struttura dello storage, con il get e set e clear. Get item della chiave ritorna il valore, set item della chiave mette una coppia chiave valore

Metodi

```
localStorage.setItem("size", "6"); salva il valore 6 sotto la chiave size con var  
size = localStorage.getItem("size"); recuperiamo il valore 6, la scoratoria e' var  
size = localStorage["size"].
```

5.5 Drag & Drop

5.6 Nuovi selettori/pseudoclassi

Nuovi selettori di

5.6.1 Selettori di attributi

Alcuni di questi selettori permettono di identificare un particolare elemento sulla base dell'attributo, e' una regola che si applica a un elemento HTML che contiene un determinato attributo o ha un certo valore per l'attributo: [attribute=^value]: seleziona tutti gli elementi che hanno un attributo